جامعة بجاية
Tasdawit n Bgayet
Université de Béjaïa

Gestion des urgences hospitalière

Réalisé par :

HAMOUCHE Amina G4
ISSAADI Yasmine G4
HADDADI Meriem G3

Encadrants:

M.BAKLI
M.OUZEGGANE
Mme.BENSLIMANE



2025/2026

DSA.2

SOMMAIRE

Introduction 3

Choix des structures de données 4

Diagramme explicatif des structures 9

Programmation modulaire 10

Description des fonctions principales 11

Difficultés rencontrées 14

Conclusion 15



1. Introduction :

La gestion des urgences hospitalières est une tâche critique qui nécessite une organisation rigoureuse et rapide. Face aux limites des méthodes manuelles, nous avons développé une application en langage C permettant de gérer efficacement le flux des patients au sein d'un service d'urgence.

Notre système assure l'enregistrement des patients à leur arrivée, le suivi de leur prise en charge, ainsi que l'enregistrement de leur sortie ou référencement vers un autre département. Pour cela, nous avons utilisé des structures de données adaptées, notamment une file d'attente pour gérer les arrivées et une liste chaînée pour les patients sous observation, avec une persistance des données assurée par des fichiers binaires.

Ce rapport présente les choix techniques effectués, une description des fonctions principales développées, ainsi que les difficultés rencontrées lors de la réalisation de ce projet.

2. choix des structures de données:

2.1 Vue d'ensemble des besoins du système:

Dans tout projet informatique, choisir la bonne structure de données est une étape essentielle. Dans notre cas, le service des urgences impose des exigences précises auxquelles le système doit répondre obligatoirement. C'est pourquoi le choix de chaque structure a été réfléchi en fonction du rôle qu'elle doit jouer dans le système.

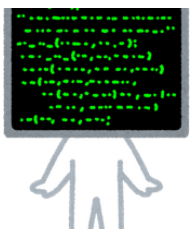
2.1.1 Les contraintes imposées par la gestion des urgences:

• L'ordre d'arrivée doit être respecté; Le premier patient arrivé doit être le premier consulté. On ne peut pas appeler n'importe qui au hasard.
• Insertions et suppressions fréquentes : les patients entrent et sortent en permanence de la file.
• Toutes les données doivent être sauvegardées: On ne doit pas perdre les informations des patients si l'application se ferme.
• Mémoire dynamique : le nombre de patients est imprévisible, on peut pas utiliser une structure de taille fixe.
• Accès non séquentiel aux patients sous observation : un patient peut quitter l'observation à n'importe quel moment, pas forcément le premier entré.

2.1.2 Pourquoi le choix des structures impacte les performances:

Le choix d'une structure de données a un impact direct sur la rapidité et la fiabilité du système, Par exemple, si on avait utilisé un tableau pour gérer la file d'attente, chaque appel d'un patient aurait obligé le programme à décaler tous les patients restants, une opération lente qui devient de plus en plus coûteuse au fur et à mesure que la file grandit.

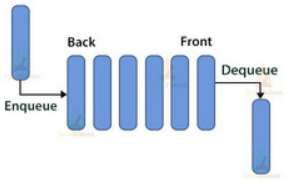
En revanche, avec une liste chaînée, le retrait d'un patient en tête se fait instantanément, sans toucher aux autres. Dans notre cas, dans un service des urgences, où chaque seconde compte une mauvaise structure de données peut ralentir tout le système. C'est pour ça chaque structure utilisée a été choisie avec soin, en tenant compte des opérations qu'elle doit effectuer et de leur fréquence.



2.2 La File d'Attente (Queue)- Gestion des Arrivées

2.2.1 Le principe FIFO et son lien avec la réalité hospitalière:

La file d'attente fonctionne selon le principe FIFO(First In, First Out), ce qui signifie que le premier patient arrivé est le premier à être consulté. Ce fonctionnement est naturellement adapté à un service des urgences, car il garantit que chaque patient est pris en charge dans l'ordre. Chaque patient reçoit un numéro de ticket lors de son arrivée, confirmant son rang dans la file.



2.2.2 Pourquoi pas un tableau? Pourquoi pas une pile?

Le tableau statique a une taille fixe définie à l'avance, ce qui pose un problème immédiat : on ne peut pas savoir combien de patients vont arriver aux urgences. De plus, chaque fois qu'un patient est retiré, il faut décaler tous les autres, ce qui ralentit le programme.

La pile fonctionne à l'envers : le dernier patient arrivé serait le premier consulté.
Ce comportement est injuste et dangereux dans un contexte médical, car un patient qui attend depuis longtemps pourrait ne jamais être appelé.

La file d'attente est donc la structure la plus adaptée. Elle respecte l'ordre d'arrivée, elle est dynamique donc sans limite de taille, et ses opérations d'ajout et de retrait sont rapides.

2.2.3 Implémentation choisie: liste chaînée dynamique

Pour implémenter la file d'attente, nous avons choisi une liste chaînée dynamique.
Elle utilise deux pointeurs : un pointeur de tête qui pointe vers le premier patient de la file, et un pointeur de queue qui pointe vers le dernier.
Grâce à cette organisation, ajouter un nouveau patient ou retirer un se fait rapidement et sans aucune limite sur le nombre de patients.

```
typedef struct t_node node;
typedef struct t_queue queue;

struct t_node
{
    patient value;
    node* next;
};

struct t_queue
{
    node* front;
    node* rear;
};
```

2.3 La Liste Chaînée — Gestion des Patients sous Observation

2.3.1 Pourquoi les patients sous observation nécessitent une structure différente:

Les patients sous observation présentent un comportement fondamentalement différent de ceux en file d'attente. Dans la file, les patients sont toujours traités dans l'ordre (le premier entré est le premier sorti). Pour l'observation, un patient peut être libéré à n'importe quel moment, indépendamment de son ordre d'admission. Son état peut s'améliorer brutalement, ou il peut être référencé en urgence vers un autre département.

2.3.2 Avantages de la liste chaînée pour ce cas précis:

- Insertion dynamique : l'ajout d'un nouveau patient en observation se fait en temps constant, sans réorganisation de la structure.
- Aucune limite de taille : le nombre de patients sous observation est imprévisible.
- Suppression flexible : un patient peut quitter l'observation sans perturber les autres.
- Parcours complet : affichage de tous les patients sous observation à tout moment.
- Mise à jour en place : modification du lit, du traitement ou du département sans réallocation.
- Mémoire optimisée : chaque nœud est alloué et libéré individuellement via malloc/free.

```
typedef struct t_node_obs     t_node_obs;
typedef t_node_obs*           obs_list;

struct t_node_obs
{
      PatientObs obs;
      obs_list   next;
};
```

2.3.3 Structure PatientObs :

Afin de centraliser toutes les informations relatives à un patient en cours d'observation, nous avons défini une structure nommée PatientObs. Cette structure constitue le cœur de chaque nœud de la liste chaînée et regroupe en un seul bloc cohérent l'ensemble des données nécessaires au suivi médical.

```
typedef struct t_patient patient;
struct t_patient
{
    int id;
    char surname[20];
    char first_name[20];
    int age;
    int ticket;
    t_statut statut;
};
```

chaque nœud contient :

- Les informations du patient : identifiant, nom, prénom, âge et statut.
- Le lit attribué : numéro et disponibilité, permettant une gestion précise des places en salle d'observation.
- Le traitement en cours : type de traitement prescrit, médicaments administrés et fréquence des soins.
- Le département concerné : service médical responsable du patient (cardiologie, pédiatrie, chirurgie, etc.).

```
typedef struct t_PatientObs PatientObs;
struct t_PatientObs
{
    patient p;
    char treatement[MAX_TREATMENT];
    int bed_number;
    t_departement departement;
};
```

Ce regroupement au sein d'une même structure présente un avantage majeur : toute opération sur un patient — qu'il s'agisse d'une consultation, d'une mise à jour ou d'une suppression — s'effectue en accédant à un nœud unique, sans avoir à parcourir plusieurs structures dispersées. Cela garantit une cohérence des données, une rapidité d'accès et une simplicité de maintenance du système.

2.4 Les Fichiers Binaires — Persistance des Données:

2.4.1 Pourquoi la persistance est indispensable:

Un système de gestion hospitalière ne peut pas perdre ses données à chaque redémarrage de l'application. La persistance des données garantit que les informations des patients, l'historique des consultations et les statistiques sont conservés de manière durable

Dans notre application, trois catégories de données doivent être persistées : les données des patients enregistrés, l'historique des consultations et des sorties, ainsi que l'état courant des patients sous observation

---

2.4.2 Binaire vs Texte : comparaison et justification du choix:

le choix entre fichier texte et fichier binaire dans notre projet n'est pas une opposition, mais une complémentarité : le binaire pour la performance et la persistance interne, le texte pour la lisibilité et la communication avec l'utilisateur.

- *Premièrement*, notre application manipule des structures C contenant les informations des patients et leur état d'observation. Avec le fichier **observation.dat**, la structure PatientObs est sauvegardée directement telle quelle en mémoire sans aucune conversion. Cela garantit une exactitude totale des données, notamment pour les valeurs numériques comme l'âge du patient, le numéro de lit ou encore l'identifiant du département concerné.

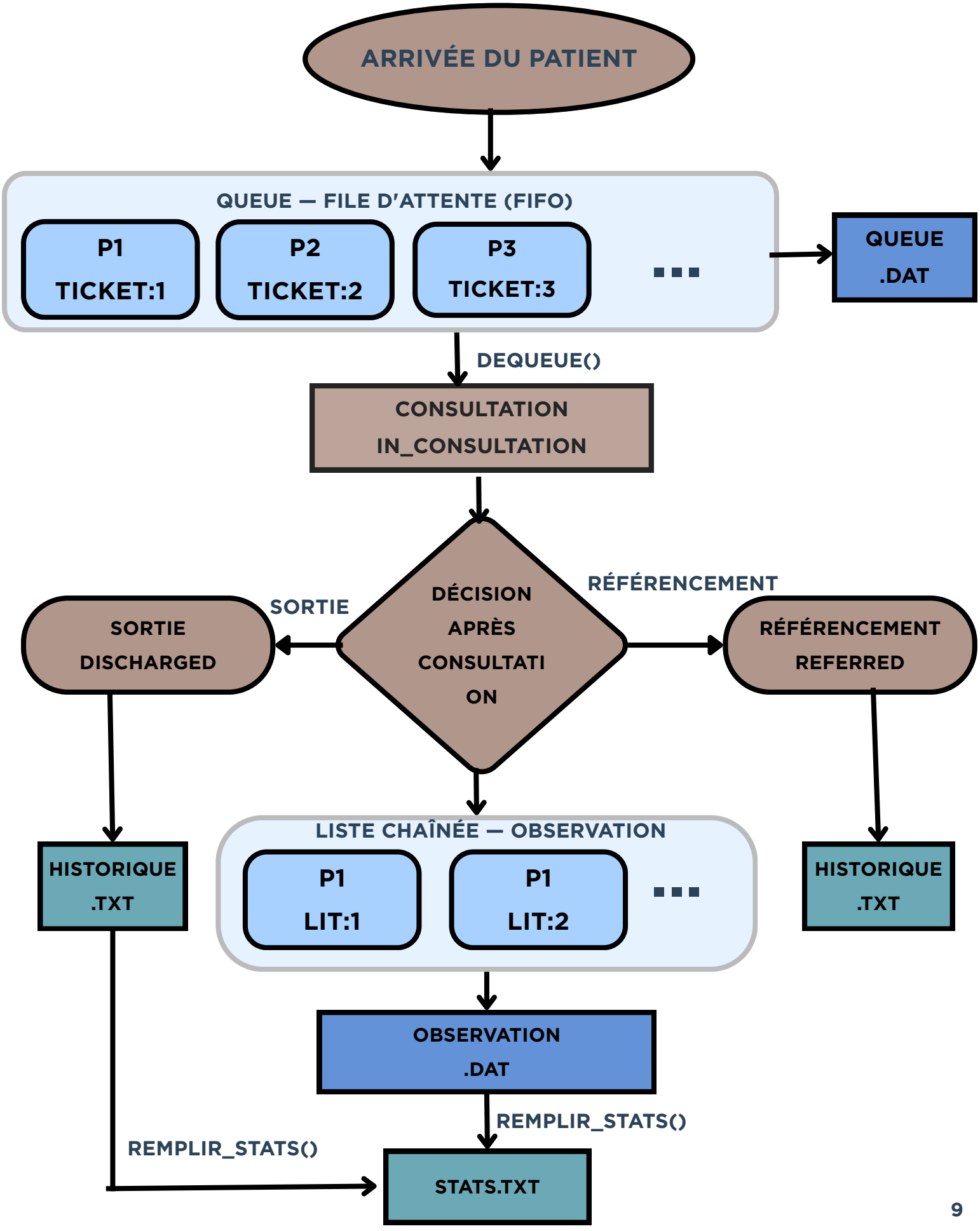
- *Deuxièmement*, le format binaire permet un chargement rapide et fiable de toutes les données au démarrage du programme. La structure PatientObs est lue telle quelle depuis le fichier et reconstituée directement en liste chaînée en mémoire, sans aucun traitement de conversion.

- *Enfin*, les fichiers texte **historique.txt** et **stats.txt** ont été choisis pour leur lisibilité directe. Le fichier **historique.txt** permet au personnel médical de consulter toutes les opérations effectuées — admissions, consultations et sorties — sans aucun outil spécial. Le fichier **stats.txt** offre un bilan clair et accessible des statistiques du service, comme le nombre de patients pris en charge ou le taux d'occupation des lits.

2.5 Tableau Récapitulatif des Choix:

	Structure utilisée:	Justification:
• **Gérer l'ordre d'arrivée des patients** -Accueil des urgences-	• file d'attente chaînée (Queue)	• Les patients arrivent de façon imprévisible et doivent être pris en charge dans l'ordre d'arrivée. • le principe FIFO garantit une équité totale. L'ajout (enqueue) et la prise en charge (dequeue).
• **Gérer les patients sous observation** -salle d'observation-	• Liste chaînée simple (ListeObservation)	• Un patient peut quitter l'observation à tout moment,quel que soit sa position dans la liste. • La liste chaînée permet une supression flexible de n'importe quel nœud sans perturber les autres.
• **Sauvegarder l'historique**	• fichier texte	• Chaque consultation,sortie ou transfert est ajouté à la fin du fichier,sans modifier les données existantes.
• **Sauvegarder les données patients**	• fichier binaire	• Les structures Patient sont sauvegardées directement en mémoire sans aucune conversion,Cela garantit une écriture rapide, une précision totale des données numériques et une compacité optimale du fichier. • Le fichier est chargé au démarrage et mis à jour à chaque modification importante.
• **Représenter un patient en file**	• structPatient -noeud chaîne-	• La structure Patient constitue le nœud de base de la file d'attente. Elle regroupe en un seul bloc l'id unique, le nom et prénom, l'âge, le numéro de ticket d'arrivée et un pointeur next vers le patient suivant.
• **Représenter un patient sous observation**	• struct PatientObs	• La structure PatientObs est le nœud central de la liste d'observation. • Elle regroupe les informations personnelles du patient, numéro du lit attribué, le traitement médical en cours, le département responsable et un pointeur next.

3.Diagramme explicatif des structures:

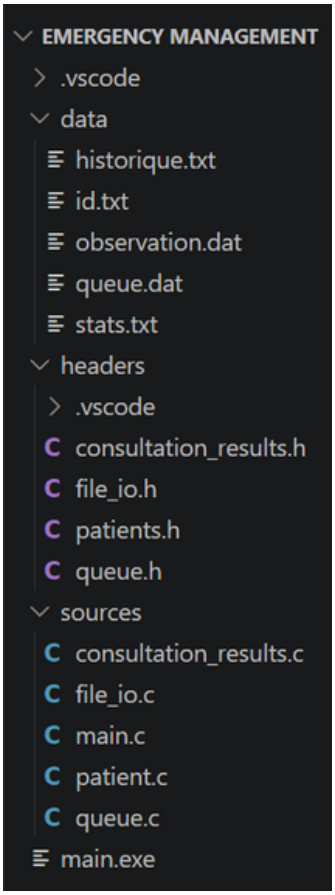




4. programmation modulaire:

- Le projet est organisé selon le principe de la programmation modulaire : chaque fonctionnalité est isolée dans un module indépendant composé d'un fichier .h (déclarations) et d'un fichier .c (implémentations).
- Le dossier headers contient les fichiers .h qui définissent les structures, les types et les signatures des fonctions. Le dossier sources contient les fichiers .c qui implémentent ces fonctions. Le dossier data contient les fichiers de persistance générés à l'exécution.
- Le projet comprend 4 modules principaux :

1. **Patients :** définit la structure patient, la création, l'affichage et la validation des données.
2. **queue** : gère la file d'attente des patients arrivants avec les opérations enqueue, dequeue, get_head.
3. **consultation_results** : gère la liste chaînée des patients hospitalisés avec refer_patient admit_patient_under_observation, discharge_patient_from_observation.
4. **file_io** : assure la persistance des données entre les sessions via des fichiers binaires et texte.
5. **main.c**: joue le rôle de coordinateur : il utilise tous les modules sans connaître leurs détails d'implémentation.



Cette organisation présente plusieurs avantages : chaque module peut être développé et testé indépendamment, les modifications dans un module n'affectent pas les autres, et le code est plus lisible et maintenable.

5. Description des fonctions principales

1.les fonctions de patient.c

- **create_patient:**
  Crée un nouveau patient à partir du nom, prénom et âge saisis. Attribue automatiquement un id et un ticket
  grâce à deux variables static qui s'incrémentent à chaque appel.Le statut initial est on_hold.
- **ticket_function:**
  Restaure le compteurs ticket au démarrage depuis le dernier patient sauvegardé, pour éviter que la numérotation recommence à 1 à chaque exécution du programme.
- **save_id**:Sauvegarde la valeur courante de la variable statique id dans un fichier texte (id.txt) à chaque fois qu'un nouveau patient est créé. Cela permet de garantir que l'identifiant ne recommence pas à zéro à chaque exécution du programme.
- **load_id:**Lit la valeur de id depuis id.txt au démarrage du programme, pour restaurer le compteur au point où il s'était arrêté lors de la session précédente.
- **display_patient:**
  Affiche toutes les informations d'un patient ticket, id, nom, prénom, âge et statut. Le statut est affiché en clair grâce à un switch.
- **correct_name:**
  Vérifie que le nom ou prénom saisi ne contient que des lettres et des espaces. Parcourt le tableau caractère par
  caractère avec isalpha() et retourne false dès qu'un chiffre ou symbole est détecté.



.

2.les fonctions de queue.c

- **is_empty:**
  (static) Vérifie si la file est vide en testant si front == NULL. Déclarée static car c'est un détail d'implémentation interne, non exposé dans le .h.
- **is_full:**
  (static) Vérifie si la mémoire est suffisante pour ajouter un nouveau nœud en tentant un malloc. Si l'allocation échoue, la file est considérée pleine. Déclarée static pour la même raison que is_empty.
- **create_queue:**
  Initialise la file en mettant front et rear à NULL. Appelée une seule fois au démarrage avant toute autre opération sur la file.
- **enqueue:**
  Alloue dynamiquement un nouveau nœud, y copie le patient, puis l'attache à la fin de la file via rear. Gère les deux cas : file vide et file non vide.
- **dequeue:**
  Supprime le premier patient de la file après sa consultation. Sauvegarde l'adresse de front, avance le pointeur, puis libère la mémoire avec free(). Met rear à NULL si la file devient vide.
- **get_head:**
  Retourne le premier patient de la file sans le supprimer. Séparée de dequeue pour respecter le principe de responsabilité unique : consulter et supprimer sont deux opérations distinctes.
- **display_queue:**
  Parcourt toute la file depuis front et affiche les informations de chaque patient via display_patient.

3.les fonctions de consultations_results.c

- **initialization_observation_list:**

Initialise la liste d'observation en mettant le pointeur à NULL. Appelée une seule fois au démarrage du programme.

- **admit_patient:**

Place un patient sous observation en créant un nœud PatientObs contenant le patient, son numéro de lit et son traitement. Le département est initialisé à autres par défaut. Le nœud est ajouté à la fin de la liste chaînée.

- **display_observation:**

Parcourt toute la liste et affiche pour chaque patient ses informations complètes : ticket, id, nom, prénom, âge, traitement et numéro de lit.

- **discharge_patient:**

Recherche le patient par son ticket dans la liste, met son statut à discharged, supprime son nœud et libère la mémoire avec free().

- **refer_patient:**

Référence un patient vers un autre département, choisi dans un menu (Cardiologie, Radiologie, Chirurgie, Autres). Le département est enregistré dans le nœud du patient, puis la référence est écrite dans historique.txt avant la suppression du nœud.

- **free_observation_list:**

Libère toute la mémoire allouée pour la liste d'observation nœud par nœud, puis remet le pointeur à NULL. Appelée à la fin du programme

- **is_bed_occupied:**

Parcourt la liste et vérifie si un numéro de lit est déjà utilisé. Retourne true si le lit est occupé, false sinon. Utilisée pour éviter d'assigner deux patients au même lit.

4.les fonctions de file-io.c:

- **sauvegarder_queue:**

Parcourt la file et écrit dans un fichier binaire uniquement les patients avec le statut on_hold. Le fichier est réécrit entièrement à chaque appel ("wb").

- **charger_queue:**

Lit le fichier binaire queue.dat au démarrage et recharge chaque patient dans la file via enqueue, pour retrouver l'état de la file de la session précédente.

- **sauvegarder_liste:**

Parcourt la liste d'observation et écrit chaque PatientObs en binaire dans un fichier. Le fichier est réécrit entièrement à chaque modification de la liste.

- **charger_observation:**

Lit le fichier observation.dat au démarrage et reconstruit la liste d'observation en mémoire en appelant admit_patient pour chaque enregistrement lu.

- **display_historique:**

Ouvre historique.txt en mode lecture et affiche son contenu ligne par ligne avec fgets.

- **remplir_stats:**

Calcule les statistiques complètes : nombre de patients en attente, sous observation, sortis, et référés par département (en additionnant les patients encore sous observation et ceux déjà référés), puis les écrit dans stats.txt.

- **display_stat:**

Ouvre stats.txt et affiche son contenu ligne par ligne.

- **discharged_patient_historique:**

Ouvre historique.txt en mode append ("a") et y ajoute une ligne avec le ticket, le nom et la prescription du patient sorti, sans effacer l'historique existant.

5.les fonctions de main.c

display_menu:

Sous-menu d'affichage qui propose 4 options :

- *Afficher la file d'attente,*
- *Afficher les patients sous observation,*
- *Afficher l'historique*
- *Afficherles statistiques.*

Chaque choix appelle directement la fonction correspondante.

```
printf("***display***\n");
printf(" \n");
printf("       [1]:display queue\n");
printf("       [2]:display patients under observatio
printf("       [3]:display historique\n");
printf("       [4]:display statistiques\n");
```

consultation_menu:

(sous_menu)C'est le cœur de la logique médicale du programme.
Après chaque consultation, le médecin choisit parmi 4 actions :

- *Sortie avec prescription* : le statut passe à discharged, la prescription est sauvegardée dans historique.txt
- *Mise sous observation* : on vérifie d'abord que le lit n'est pas déjà occupé grâce à is_bed_occupied, puis on admet le patient
- Sortie de l'observation : supprime le patient de la liste et met à jour le fichier
- Référencement : Référencement : le médecin choisit le département de destination dans un menu à choix (1-4), le patient est transféré, le département est enregistré, et l'opération est ajoutée à l'historique.

```
printf("***Record the consultation results***\n");
printf(" \n");
printf("       [a]:discharge with prescreption \n");
printf("       [b]:place under observation\n");
printf("       [c]:Release a patient from observation\n");
printf("       [d]:refer to another departement\n");
```

MAIN

Coordonne l'ensemble du système. Au démarrage, il recharge automatiquement la file et la liste d'observation depuis les fichiers binaires, et restaure les compteurs de tickets et d'id via ticket_id en passant les valeurs du dernier patient sauvegardé dans la file.
Ensuite une boucle do-while tourne jusqu'à ce que l'utilisateur choisisse de quitter, avec 4 fonctionnalités principales :

- *Enregistrement.*
- *Appel du prochain patient.*
- *Résultat de la Consultation.*
- *Affichage.*

```
printf("*** Hospital Emergency Management System***");
printf("\n");
printf("       [1]:Register a patient's arrival\n");
printf("       [2]:call the next patient\n");
printf("       [3]:Record the consultation results\n");
printf("       [4]:display\n");
printf("       [5]:exit\n");
```

6. Difficultés rencontrées:

1. Persistance des compteurs ticket et id entre les exécutions:

la difficulté rencontrée était la gestion du numéro de ticket lors d'un redémarrage du programme. Si la queue est vide (début de journée), le ticket repart naturellement de zéro. En revanche, si la queue contient encore des patients — comme dans le cas d'une coupure d'électricité en plein milieu de journée — il faut absolument reprendre depuis le dernier ticket attribué, pour éviter que deux patients se retrouvent avec le même numéro. La solution était de lire le ticket directement depuis q.rear au chargement. Contrairement au ticket, le id ne se réinitialise jamais, même en début de journée, car il est unique à chaque patient pour toute la durée de vie du système.

2.Patient en consultation non initialisé:

Une difficulté rencontrée était que le patient récupéré pour la consultation dépend d'une étape précédente — l'appel du patient. Si le médecin tente d'enregistrer les résultats d'une consultation sans avoir appelé de patient au préalable, la variable p sera non initialisée, ce qui peut mener à un comportement imprévisible. Cette situation reste un cas non géré dans notre implémentation actuelle.



3.La cohérence des données entre les fichiers:

- Quand le programme redémarre, il fallait recharger correctement la queue et la liste d'observation
- Les tickets et IDs devaient correspondre exactement aux données sauvegardées

4.La conception des cas de test:

L'une des difficultés était de trouver des tests capables de faire apparaître les erreurs. Les tests simples fonctionnaient facilement, mais les cas limites étaient difficiles à imaginer, comme appeler un patient quand la file est vide, libérer un patient inexistant, ou assigner un lit déjà occupé. Il fallait se mettre dans la peau d'un utilisateur qui utilise mal le système pour découvrir toutes les failles.

7. Conclusion

Ce projet nous a permis de développer une application complète de gestion des urgences hospitalières en langage C, répondant à l'ensemble des objectifs fixés. Le système permet d'assurer l'enregistrement des patients, le suivi de leur prise en charge et l'enregistrement de leur sortie ou référencement vers un autre département.

La réalisation de ce projet nous a permis de mettre en pratique les concepts fondamentaux du module DSA-2, notamment l'utilisation des structures de données dynamiques telles que la file d'attente et la liste chaînée, la gestion de la mémoire avec malloc et free, ainsi que la manipulation des fichiers binaires pour la persistance des données.

Enfin, ce projet pourrait être amélioré dans le futur en ajoutant une interface graphique plus conviviale, un système de priorité selon la gravité de l'état du patient, ainsi qu'une base de données pour remplacer les fichiers binaires et garantir une meilleure gestion des données.

Nous tenons à remercier sincèrement nos encadrants ***M. BAKL***I, ***M. OUZEGGANE*** et ***Mme. BENSLIMANE*** pour leur disponibilité, leurs précieux conseils et leur accompagnement tout au long de la réalisation de ce projet.

Merci de votre attention

Gestion des urgences hospitalière

DSA.2 | 2025/2026
Université A.Mira — Béjaïa

